



Documentação Técnica do
Zion System – Sistema de Gestão de Cadastro de Dados
e Gerenciamento de Vendas

3º Período BSI

Docente: Tiago Batista Pedra

Discentes: Renan Carlos V. Silva,

Sávio Renner P. Duarte,

Vitor Hugo C. Sousa.

Curitiba,

2026

1 - Resumo do Projeto e Problemática Abordada.

O Zion System é um sistema de software integrado e focado de gestão de dados empresariais; ele foi desenvolvido para a empresa fictícia Bulldogs PC, uma empresa especializada em venda de peças para computadores gamers, mas que sofre com a falta de avaliação de métricas de vendas e organização cadastral.

O projeto surge como solução estratégica para um problema crítico enfrentado pela organização: a ausência de um sistema centralizado para gerenciar informações de clientes, fornecedores, produtos e vendas comerciais, bem como apresentar (de forma dinâmica e consolidada) relatórios referentes aos produtos ofertados, lucros, e peças com grande aderência de compra, de forma que, ao analisar os resultados dispostos no relatório, a empresa seja capaz de tomar decisões estratégicas, ofertar benefícios aos clientes de alto valor, e aferir o lucro final.

2 – Funcionalidades Implementadas.

Classe de Clientes: funcionalidade de clientes permite o cadastro completo com atributos como nome, endereço, CPF ou CNPJ, e-mail, telefone e status (Online, Offline, Sleepy). O sistema mantém histórico de clientes inativos (offline) e oferece busca em tempo real por nome ou ID, facilitando a localização de informações para consultas comerciais.

Classe de Fornecedores: Gestão completa de fornecedores com registro de múltiplas peças, quantidades e valores unitários. O sistema gerencia o status de atividade na empresa (Online para fornecedores ativos, Offline para encerrados, Sleepy para em trâmite) e permite alteração parcial ou total de dados através das rotas PATCH e PUT, respectivamente.

Classe de Peças Cadastro de peças com atributos de identificação, descrição, valor unitário, quantidade em estoque e fornecedor associado. A interface permite visualização em tabela com opções de edição e exclusão, facilitando a manutenção do catálogo de produtos e o controle de estoque.

Classe de Vendas: são as transações de venda com rastreamento de cliente, fornecedor, peças adquiridas, quantidades, valores e data da transação. Os dados de venda alimentam o módulo de relatórios, permitindo análise de padrões de consumo e contribuindo para decisões estratégicas de marketing e investimento.

Guia de Cadastro: nesta aba, vinculada ao Front-End do Sistema, encontram-se as opções para o cadastro de cliente, fornecedor, peças e vendas (vinculadas ao ID do cliente); esta funcionalidade consome a rota POST na API. Ela está vinculada ao JavaScript e o HTML do código.

Guia Listagem: nesta aba é possível encontrar clientes, fornecedores, peças e vendas cadastrados, também é possível editar o valor de venda (ocorrerá o consumo da rota PATCH), pesquisar um cliente por ID (Ocorrera o consumo da rota GET), além da exclusão de dados (Ocorrera o consumo da rota Delete).

Guia de Dashboard e Relatórios Interface analítica que apresentam 14 informações: faturamento total, custo total, margem de lucro, ticket médio por cliente, volume total de peças vendidas, peça mais vendida, peça menos vendida, quantidade de clientes ativos, fornecedor mais utilizado, lucro por peça, lucro por cliente, faturamento por fornecedor, peça mais lucrativa e margem de venda percentual.

3 - Descrição das Funcionalidades com API

3.1 Arquitetura e Pilares de Programação Orientada a Objetos

O Zion System foi desenvolvido utilizando C# com fundamentação em quatro pilares de Programação Orientada a Objetos. A herança é implementada através da classe base DadosBase, que centraliza atributos comuns (Nome/Razão Social, Endereço, CPF/CNPJ, E-mail e Telefone) reutilizados por Clientes e Fornecedores e Colaboradores. O encapsulamento garante que todos os atributos sejam declarados com modificadores de acesso apropriados e acessados via propriedades get e set.

3.2 Integração API REST e ASP.NET Core

A aplicação implementa uma API RESTful utilizando ASP.NET Core 9.0, fornecendo endpoints para operações CRUD (Create, Read, Update, Delete) em todas as entidades. A comunicação entre frontend (HTML/CSS/JavaScript) e backend (Csharp ou C#) ocorre via requisições HTTP (GET, POST, PUT – em desenvolvimento, PATCH, DELETE). O Program.cs configura os serviços de controladores (CORS). A interface web é construída com

HTML – estrutura, CSS – responsividade e JavaScript - interação, para ser funcional e dinâmico.

O arquivo `index.html` estrutura as abas de Cadastro, Listagem e Dashboard. O arquivo `stylezion.css` fornece estilização com suporte a responsividade, também alinhando as guias, formas e cores da página. O arquivo `zion.js` abrange a lógica de interação, incluindo funções para validação de formulários, atualização dinâmica de tabelas e gerenciamento de estado da interface.

3.3 Funcionalidades Técnicas Específicas

Busca: a procura por clientes implementa filtros em tempo real por nome ou ID, para otimizar performance. Os formulários de cadastro validam dados antes do envio e resetam o seletor de tipo após a conclusão. As tabelas de listagem atualizam dinamicamente via DOM manipulation, com botões de Editar e Deletar vinculados a funções que chamam as rotas PUT/PATCH e DELETE respectivamente. A função “`editarItem()`” cria um formulário de edição com dados existentes, permitindo alteração parcial (PATCH) dos registros.

Obs.: O armazenamento de Dados no `program.cs` utiliza banco em memória (RAM), funcionando como ambiente de teste. Esta abordagem permite desenvolvimento rápido e validação de funcionalidades sem dependência de banco de dados externo. Para produção, a arquitetura foi projetada para facilitar migração para um banco de dados futuro (SQL Server ou PostgreSQL.).

4 – EndPoints Utilizados no Sistema

A API do Zion System expõe os seguintes endpoints:

RESTful para operações em cada entidade do sistema. Cada endpoint segue o padrão REST com verbos HTTP e retorna respostas em formato JSON.

Clientes GET /clientes — Recupera lista completa de clientes cadastrados com todos os atributos (ID, nome, endereço, CPF, e-mail, telefone, data de cadastro, status).

GET /clientes/{id} — Pesquisa um cliente específico pelo ID, retornando seus dados completos.

GET /fornecedores/{id} — Pesquisa fornecedor específico pelo ID com todos seus dados e peças associadas.

POST /clientes — Cadastra um novo cliente. Recebe objeto JSON com nome, endereço, CPF, e-mail, telefone. O sistema gera automaticamente o ID.

PUT /clientes/{id} — Atualiza todos os dados de um cliente existente. Requer todos os campos do cliente no corpo da requisição. (rota não concluída - em desenvolvimento).

PATCH /clientes/{id} — Atualiza parcialmente um cliente, permitindo modificação de campos específicos sem afetar outros atributos.

DELETE /clientes/{id} — Remove um cliente do sistema pelo seu ID. Fornecedores GET /fornecedores — Retorna lista completa de fornecedores com informações de peças, quantidades, valores unitários, valor total, data, status e ID.

POST /fornecedores — Cadastra novo fornecedor com nome, endereço, CNPJ, e-mail, telefone, lista de peças, quantidades e valores unitários. Sistema calcula valor total automaticamente.

PUT /fornecedores/{id} — Atualiza todos os atributos do fornecedor incluindo peças, quantidades e valores.

PATCH /fornecedores/{id} — Modifica parcialmente dados do fornecedor, frequentemente utilizado para alterar preço de peça específica ou status.

DELETE /fornecedores/{id} — Remove fornecedor do sistema. Peças GET /pecas — Retorna catálogo completo de peças com ID, descrição, valor unitário, quantidade em estoque e fornecedor.

GET /pecas/{id} — Pesquisa peça específica pelo ID com todos seus atributos. POST /pecas — Cadastra nova peça com descrição, valor unitário, quantidade inicial e fornecedor associado.

PUT /pecas/{id} — Atualiza todos os dados da peça (rota em desenvolvimento / conclusão).

PATCH /pecas/{id} — Atualiza parcialmente a peça.

DELETE /pecas/{id} — Remove peça do catálogo.

Vendas GET /vendas — Recupera histórico completo de transações de venda com cliente, fornecedor, peças, quantidades, valores e data.

GET /vendas/{id} — Pesquisa venda específica pelo ID com detalhamento completo da transação.

POST /vendas — Registra nova venda. Recebe cliente ID, fornecedor ID, lista de peças com quantidades. Sistema calcula valores totais automaticamente. PUT /vendas/{id} — Atualiza registro completo de venda.

PATCH /vendas/{id} — Modifica parcialmente dados da venda. DELETE /vendas/{id} — Remove registro de venda do histórico. Dashboard e Relatórios

GET /dashboard — Retorna objeto JSON contendo 14 métricas calculadas dinamicamente: faturamento total, custo total, margem de lucro, ticket médio, volume de peças vendidas, peça mais vendida, peça menos vendida, quantidade de clientes ativos, fornecedor mais utilizado, lucro por peça, lucro por cliente, faturamento por fornecedor, peça mais lucrativa e margem de venda percentual.

Referências de Construção:

Microsoft Learn — Tutorial: Criar uma Minimal API com ASP.NET Core. Acesso em: mai. 2026.

Protocolo HTTP - o que é uma API: Solução de problemas em software e Desenvolvimento de Api Modelo. Acesso em: mai.2026

